Exercise 1

Telecommunication Systems I

Technical University of Crete
School of Electrical and Computer Engineering
Instructor: Athanasios P. Liavas
Group Members: Georgios Markantonakis 2021030028 & Spyridon Alygizakis 2021030026

16 March 2026

Time required to complete the exercise: 6.5 hours.

A.1

Statement: Construct SRRC pulses $\phi(t)$. Indicative values: $T = 10^{-3}$sec over $= 10$, $T_s = \frac{T}{\text{over}}$ $A = 4$, and roll-off factor $a = 0, 0.5, 1$ (here A denotes half the length of the truncated pulse measured in symbol periods). Plot the pulses on the same figure (a common plot) with an appropriate time axis, for $T = 10^{-2}$ sec, over $= 10$, $A = 4$, and $a = 0, 0.5, 1$. What do you observe about the rate at which the pulse amplitude "decays" as the absolute time t increases, with respect to the value of a? Which pulse decays faster?

Figure:

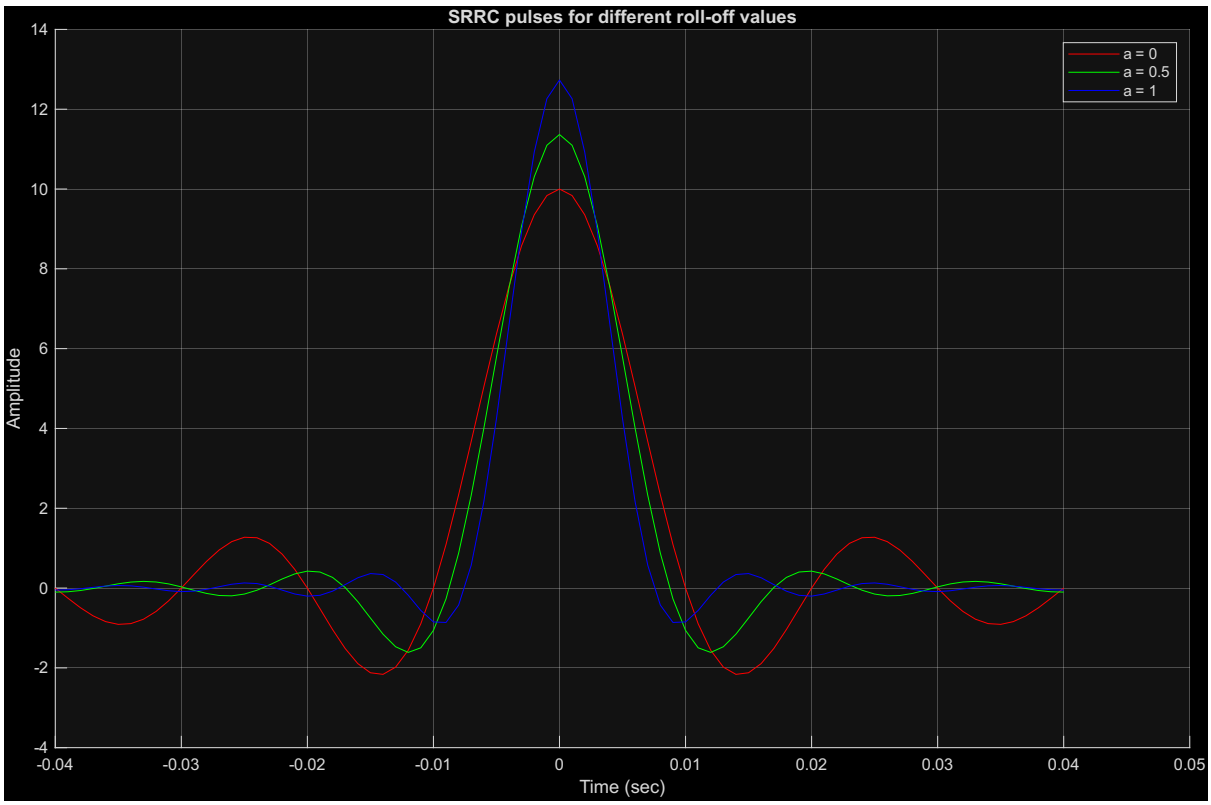

Figure 1: SRRC pulses for $T = 10^{-2}$ sec, over $= 10$, $A = 4$, and $a = 0, 0.5, 1$.

MATLAB Code:

```
% Parameters
T = 10^(-2);
over = 10;
Ts = T/over;
A = 4;
a = [0, 0.5, 1];
colors = ['r', 'g', 'b'];
labels = {'a = 0', 'a = 0.5', 'a = 1'};


%A.1
figure(1);
hold on;
for i = 1:length(a)
[pulse, t] = srrc_pulse(T, over, A, a(i));
plot(t, pulse, colors(i));
end

xlabel('Time (sec)')
ylabel('Amplitude')
grid on;
title('SRRC pulses for different roll-off values');
legend(labels);

```

Explanation: As the roll-off factor a increases, the pulse amplitude decays much faster as absolute time t increases. The pulse with $a = 1$ decays the fastest, exhibiting the smallest side lobes in the time domain.

A.2

Statement: Using the functions fft and fftshift, compute the Fourier transforms $\Phi(F)$ of the pulses from the previous step at N_f equally spaced frequency points over $[-\frac{F_s}{2}, \frac{F_s}{2})$ (e.g., $N_f = 1024, 2048$). Plot the energy spectral density $|\Phi(F)|^2$ of these pulses (a) on a common plot, (b) on a common semilogy (note that semilogy lets you study the values of $|\Phi(F)|^2$ over ranges where they are very small, which cannot be done easily with a linear plot).

Figures:

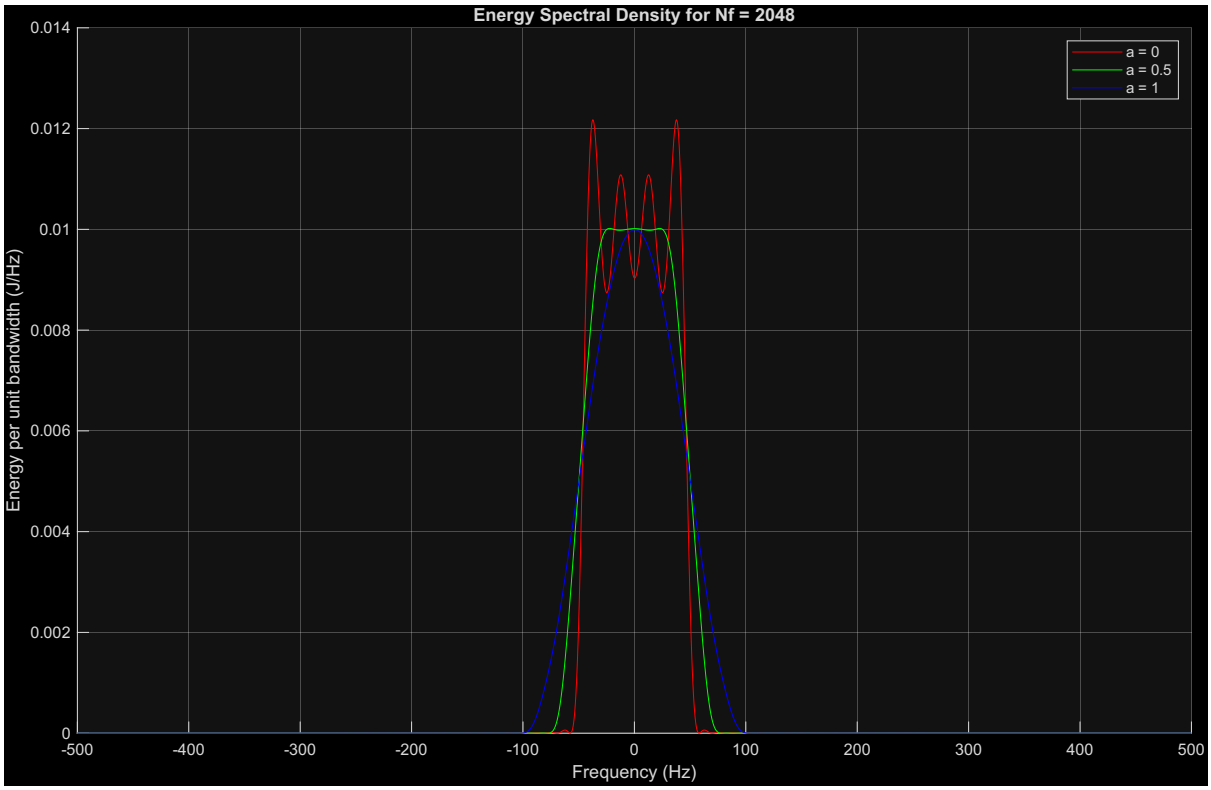

Figure 2: Energy Spectral Density (Linear) for $N_f = 2048$.

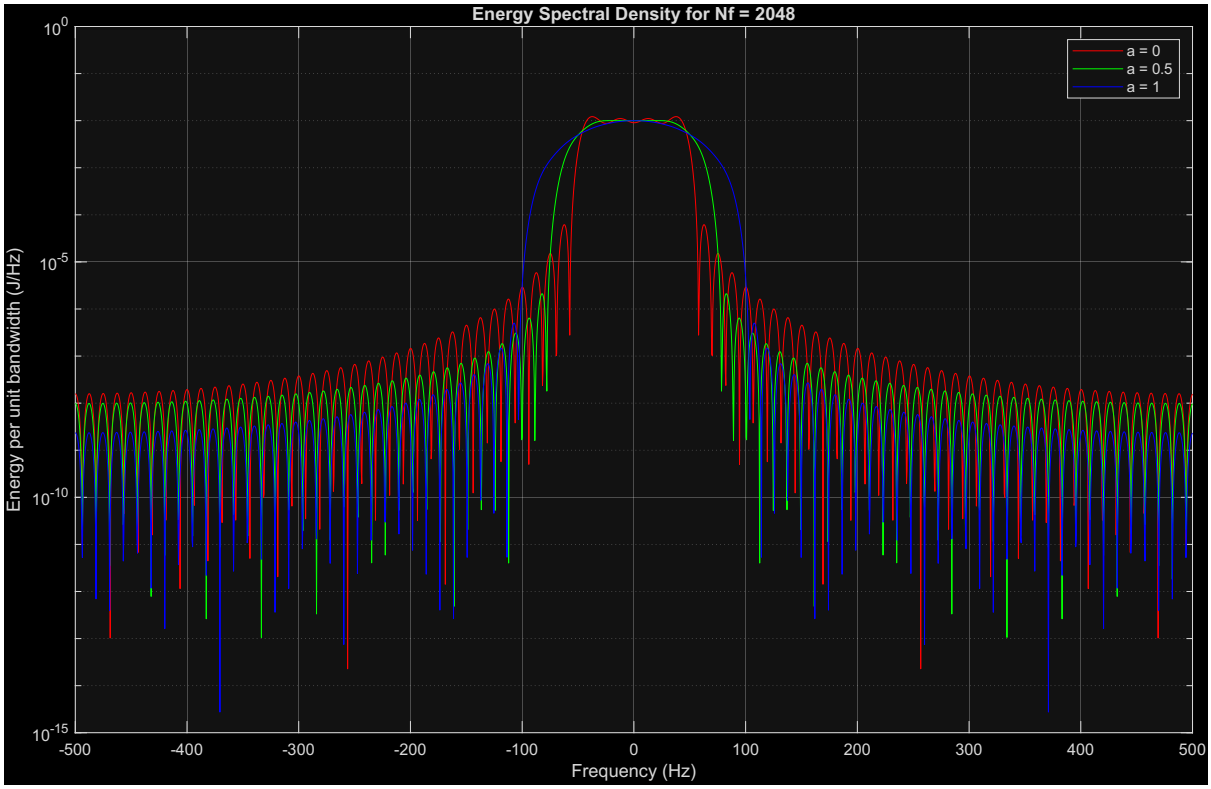

Figure 3: Energy Spectral Density (Semilogy) for $N_f = 2048$.

MATLAB Code:

```
%A.2 A
Nf = [1024, 2048];
for i = 1: length(Nf)
figure();
for j = 1 : length(a)
[pulse, ~] = srrc_pulse(T, over, A, a(j));
Fs = 1./Ts;
f_axis = linspace(-Fs/2, Fs/2, Nf(i)); % Create frequency axis for plotting
Freq_pulse = fftshift(fft(pulse, Nf(i))) .* Ts; % Fast fourier transform of srrc pulse
pulse_energy = abs(Freq_pulse).^2;
hold on;
plot(f_axis, pulse_energy, colors(j));
end

xlabel('Frequency (Hz)');
ylabel('Energy per unit bandwidth (J/Hz)');
grid on;
title(sprintf('Energy Spectral Density for Nf = %i', Nf(i)));
legend(labels);
hold off;
end;


%A.2 B
for i = 1: length(Nf)
figure();
for j = 1 : length(a)
[pulse, ~] = srrc_pulse(T, over, A, a(j));
Fs = 1./Ts;
f_axis = linspace(-Fs/2, Fs/2, Nf(i));
```

```
    Freq_pulse = fftshift(fft(pulse, Nf(i))) .* Ts;
    pulse_energy = abs(Freq_pulse).^2;
    semilogy(f_axis, pulse_energy, colors(j));
    hold on;
    end

    xlabel('Frequency (Hz)');
    ylabel('Energy per unit bandwidth (J/Hz)');
    grid on;
    title(sprintf('Energy Spectral Density for Nf = %i', Nf(i)));
    legend(labels);
    hold off;
    end

```

A.3

Statement: The theoretical bandwidth of infinite-duration pulses is $BW = \frac{1+a}{2T}$. Compute the theoretical bandwidth for each of the three pulses. In practice, since truncated pulses have (theoretically) infinite bandwidth, a more practical definition of bandwidth is needed. On the common semilogy plot from question A.2, draw a horizontal line with value c (e.g., $c = \frac{T}{10^3}$) and consider values below this line to be "practically zero." Under this definition, what is the approximate bandwidth of the three pulses (the "Zoom in" feature will be helpful)? Which pulse is more bandwidth-efficient? How does the bandwidth change if $c = \frac{T}{10^5}$? In this case, which pulse is more efficient?

Figure:

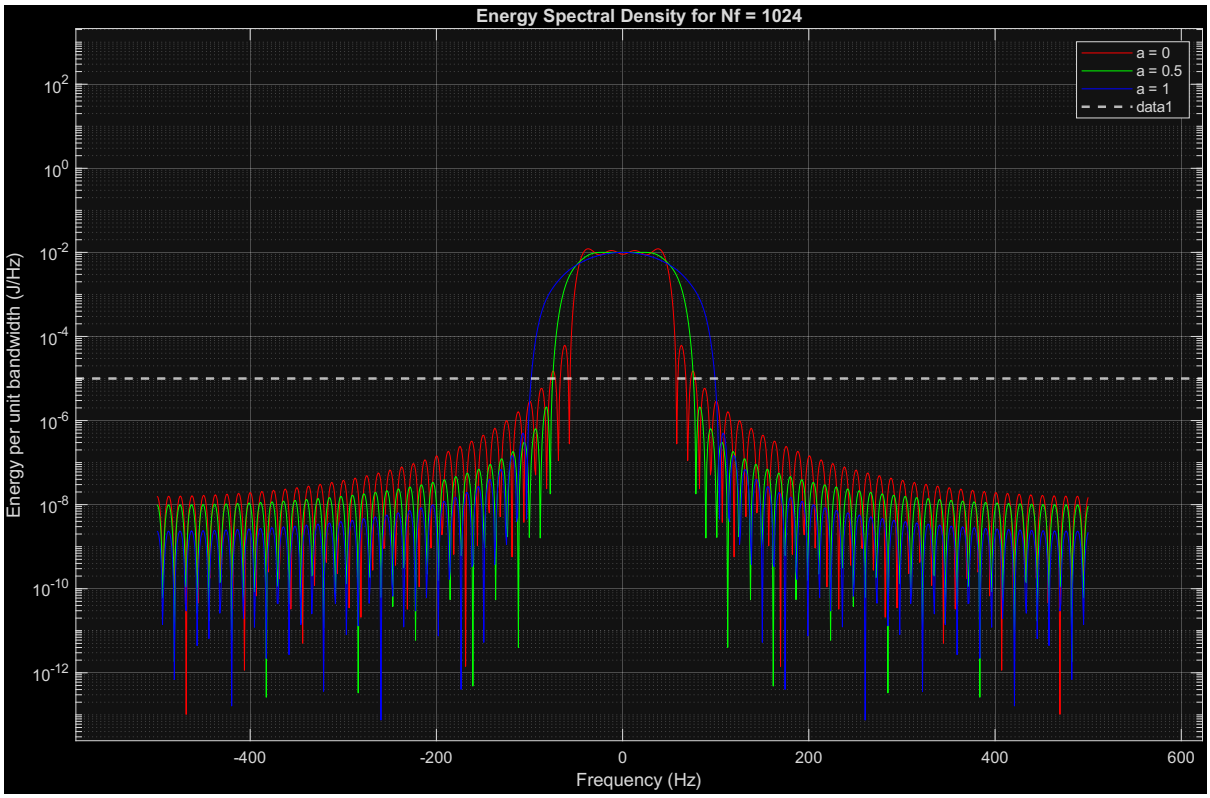

Figure 4: Energy Spectral Density for $N_f = 1024$ with $C = T/10^3$ cutoff

MATLAB Code:

```
%A.3
figure(4);
c = T / 10^3;
yline(c, '--w', 'LineWidth', 1.5);

```

Explanation: Theoretical bandwidth is calculated as $BW = \frac{1+a}{2T}$. For $T = 10^{-2}$ sec:

- $a = 0 \Rightarrow BW = 50$ Hz
- $a = 0.5 \Rightarrow BW = 75$ Hz
- $a = 1 \Rightarrow BW = 100$ Hz

Using the $c = \frac{T}{10^3}$ threshold on the semilogy plot, the pulse with $a = 0$ appears most bandwidth-efficient as it contains its energy within the narrowest main lobe. However, at a stricter threshold of $c = \frac{T}{10^5}$, pulses with higher roll-off (e.g., $a = 1$) become more efficient because their out-of-band energy decays far more rapidly, dropping below the cutoff faster than the $a = 0$ pulse.

B.1

Statement: For $T = 10^{-2}$, $A = 4$, $a = 0, 0.5, 1$, and $k = 0, 1, ..., 2A$ (you will obtain corresponding results for negative k) 1. plot $\phi(t)$ and $\phi(t-kT)$ on the same plot, 2. plot the product $\phi(t)\phi(t-kT)$, 3. numerically approximate the integral of the product $\phi(t)\phi(t-kT)$ using the method described in the notes. Include in your report the results of steps 1 and 2 for $a = 0.5$ and $k = 0, 1, 2, 3$. Include in your report the values of the integrals from step 3 for $a = 0, 0.5, 1$ and $k = 0, 1, 2, 3$, and try to explain them.

Figures:

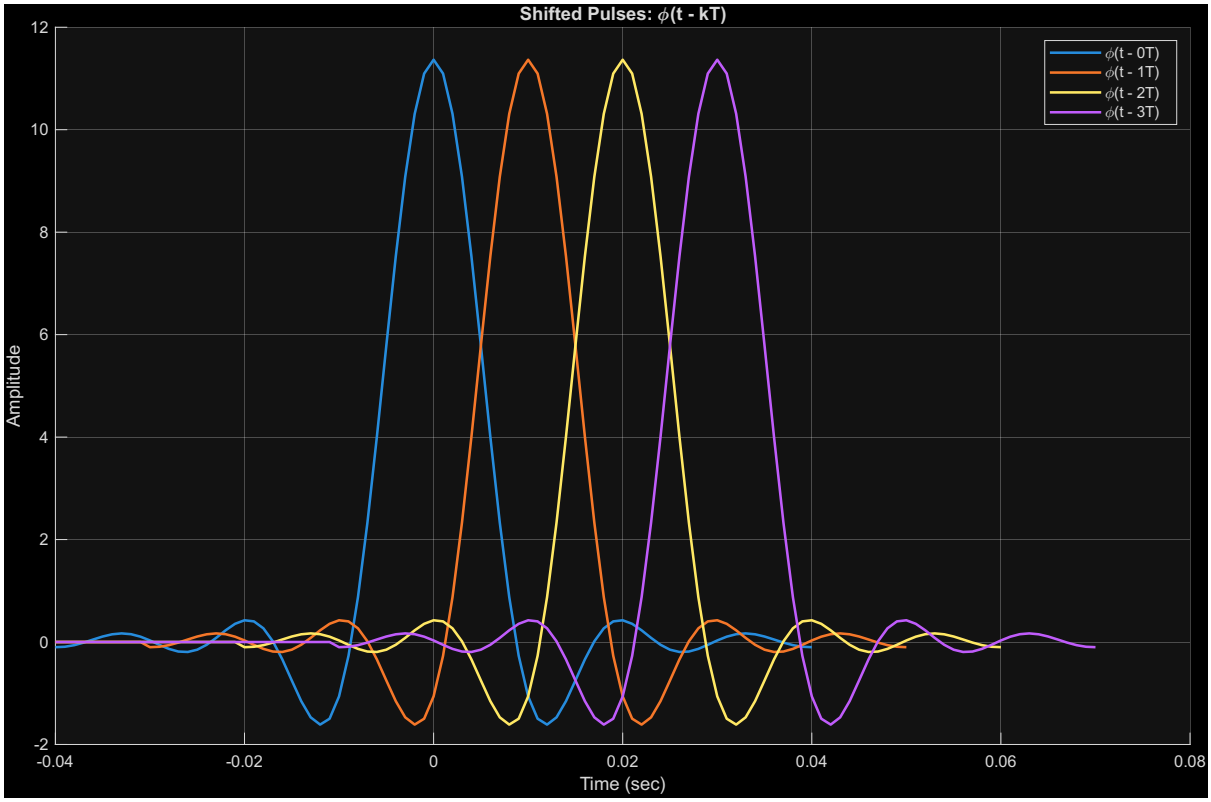

Figure 5: Shifted Pulses for $a = 0.5$ and $k = 0, 1, 2, 3$.

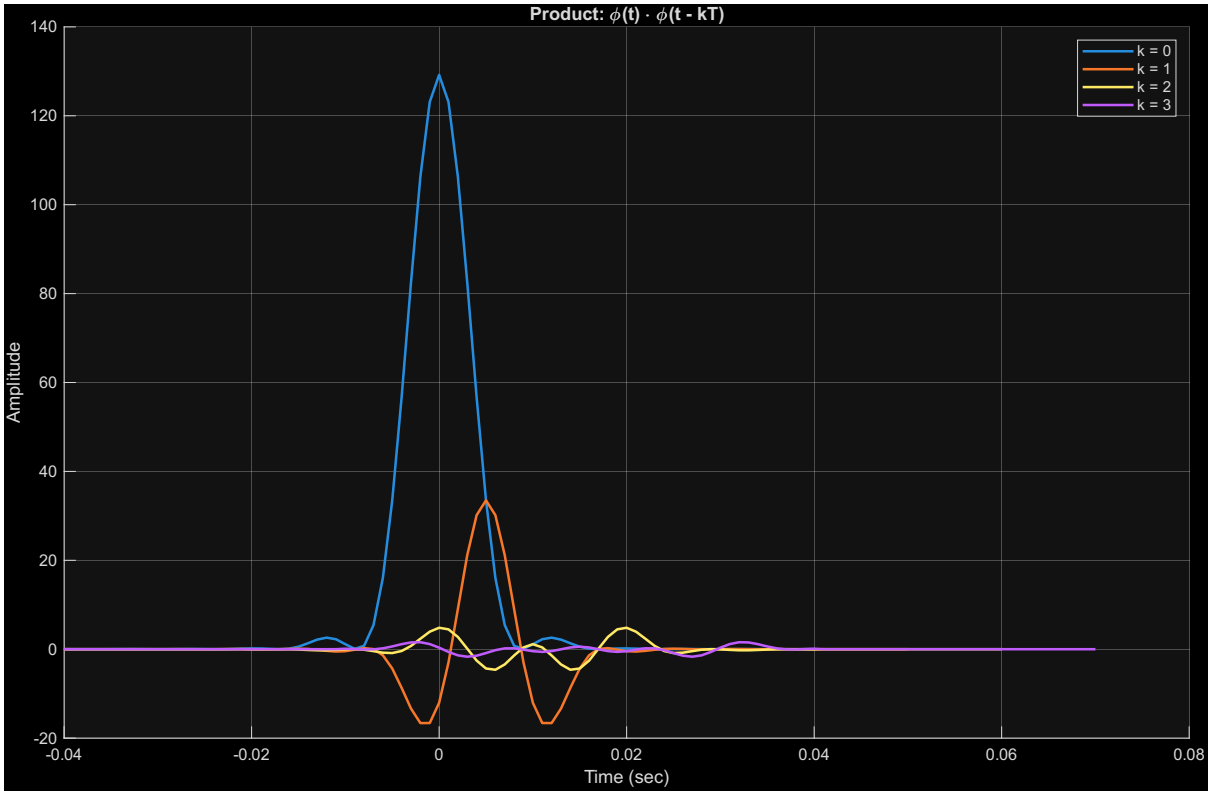

Figure 6: Shifted Pulses and their Product for $a = 0.5$ and $k = 0, 1, 2, 3$.

MATLAB Code:

```
%B.1 1, 2
k_vals = 0:1:3;
[pulse, t] = srrc_pulse(T, over, A, a(2));

figure(6); hold on; grid on;
title('Shifted Pulses: \phi(t - kT)');
xlabel('Time (sec)'); ylabel('Amplitude');

figure(7); hold on; grid on;
title('Product: \phi(t) \cdot \phi(t - kT)');
xlabel('Time (sec)'); ylabel('Amplitude');

colors = lines(length(k_vals)); % Distinct colors for each k value

for i = 1:length(k_vals)
k = k_vals(i);

shift_samples = k * over;

%Align original pulse by padding zeros at the end
pulse_orig_aligned = [pulse, zeros(1, shift_samples)];

%Shifted version phi(t - kT) by padding zeros at the beginning
pulse_shift_aligned = [zeros(1, shift_samples), pulse];

t_aligned = t(1) + (0:(length(pulse_orig_aligned)-1)) * Ts;

%phi(t) * phi(t - kT)
product_sig = pulse_orig_aligned .* pulse_shift_aligned;

figure(6);
```

```
    plot(t_aligned, pulse_shift_aligned, 'Color', colors(i,:), 'LineWidth', 1.5, ...
    'DisplayName', sprintf('\\phi(t - %dT)', k));

    figure(7);
    plot(t_aligned, product_sig, 'Color', colors(i,:), 'LineWidth', 1.5, ...
    'DisplayName', sprintf('k = %d', k));
    end

    figure(6); legend('show'); hold off;
    figure(7); legend('show'); hold off;

```

Table 1: Numerically approximated integrals of the product $\phi(t)\phi(t-kT)$

Shift (k)	**Roll-off** ($a = 0.0$)	**Roll-off** ($a = 0.5$)	**Roll-off** ($a = 1.0$)
$k = 0$	0.974749	0.999876	0.999969
$k = 1$	0.029027	0.000022	-0.000047
$k = 2$	-0.034885	0.000333	-0.000082
$k = 3$	0.046111	-0.000341	-0.000203

Explanation: The integral of $\phi(t)\phi(t-kT)$ evaluates to ≈ 1 for $k = 0$ (total pulse energy) and values approaching 0 for $k \neq 0$. This confirms the approximate orthonormality of the truncated SRRC pulses with respect to shifts by kT. The approximation improves (integrals closer to exactly zero for $k \neq 0$) as $a \to 1$ and as the truncation factor A increases.

C.1

Statement: Generate N bits b_i, for $i = 0, ..., N-1$ (e.g., $N = 50, 100$), using the command $b = (sign(randn(N, 1)) + 1)/2$

C.2

Statement: The baseband 2-PAM system is implemented as follows. 1. Write a function X = bits_to_2PAM(b); which takes as input the bit sequence b and outputs the sequence of 2-PAM symbols X using the mapping: $0 \longrightarrow +1$, $1 \longrightarrow -1$. 2. Simulate the signal $X_\delta(t) = \sum_{k=0}^{N-1} X_k \delta(t - kT)$, using the command X_delta = 1/T_s * upsample(X, over); Define the appropriate time axis and plot $X_\delta(t)$. 3. Create a truncated SRRC pulse $\phi(t)$ (using the parameters above) and simulate the convolution $X(t) = X_\delta(t) \oplus \phi(t)$ Construct the appropriate time axis and plot $X(t)$. 4. Assuming an ideal channel, the receiver input is $X(t)$. Simulate the convolution $Z(t) = X(t) \oplus \phi(-t)$. Plot $Z(t)$ over the corresponding time axis and find the values it takes at time instants kT, for $k = 0, ..., N-1$ Relate these values to X_k, Can you explain the phenomenon? A graphical way to compare $Z(kT)$ with X_k for $k = 0, ..., N-1$, is hold on the plot of $Z(t)$ and execute $stem([0 : N-1] * T, X)$; where X is the vector of symbols X_k, $k = 0, ..., N-1$ What do you observe? Can you explain the phenomenon?

Figures:

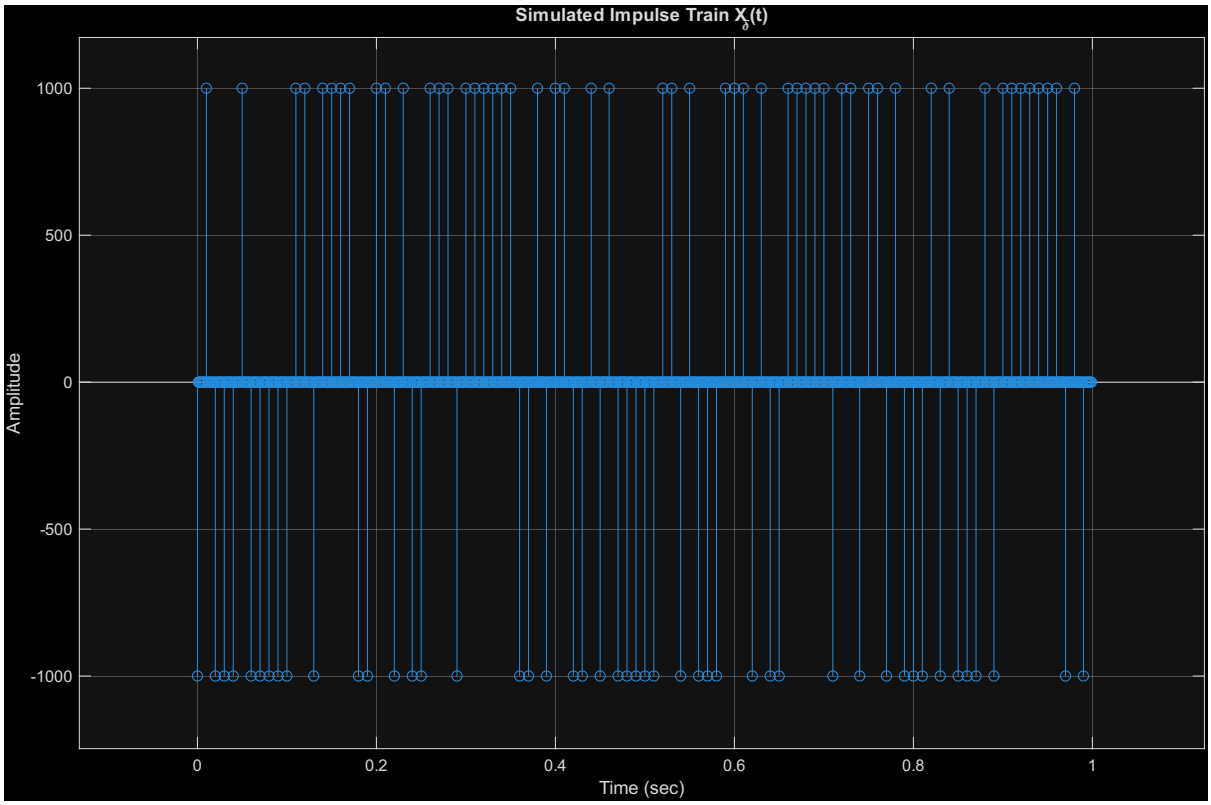

Figure 7: X_d signal

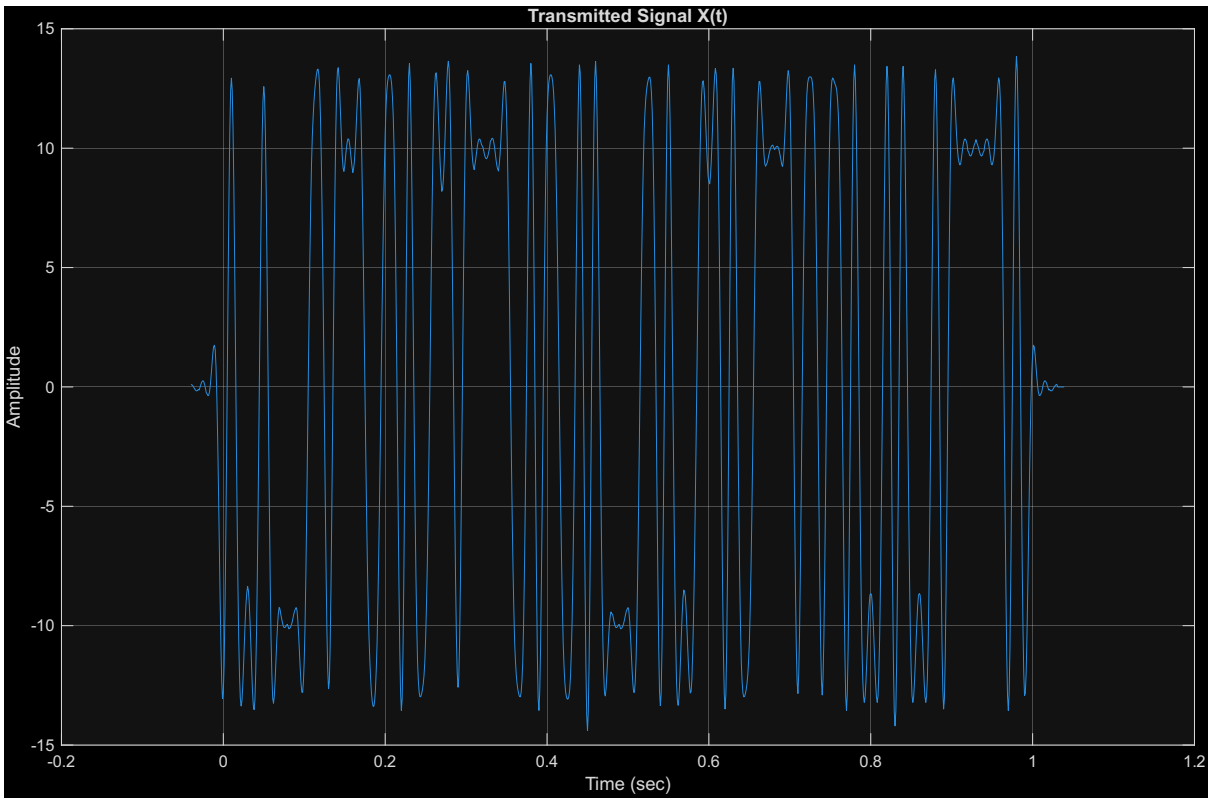

Figure 8: Simulated $X(t)$ signal from convolution.

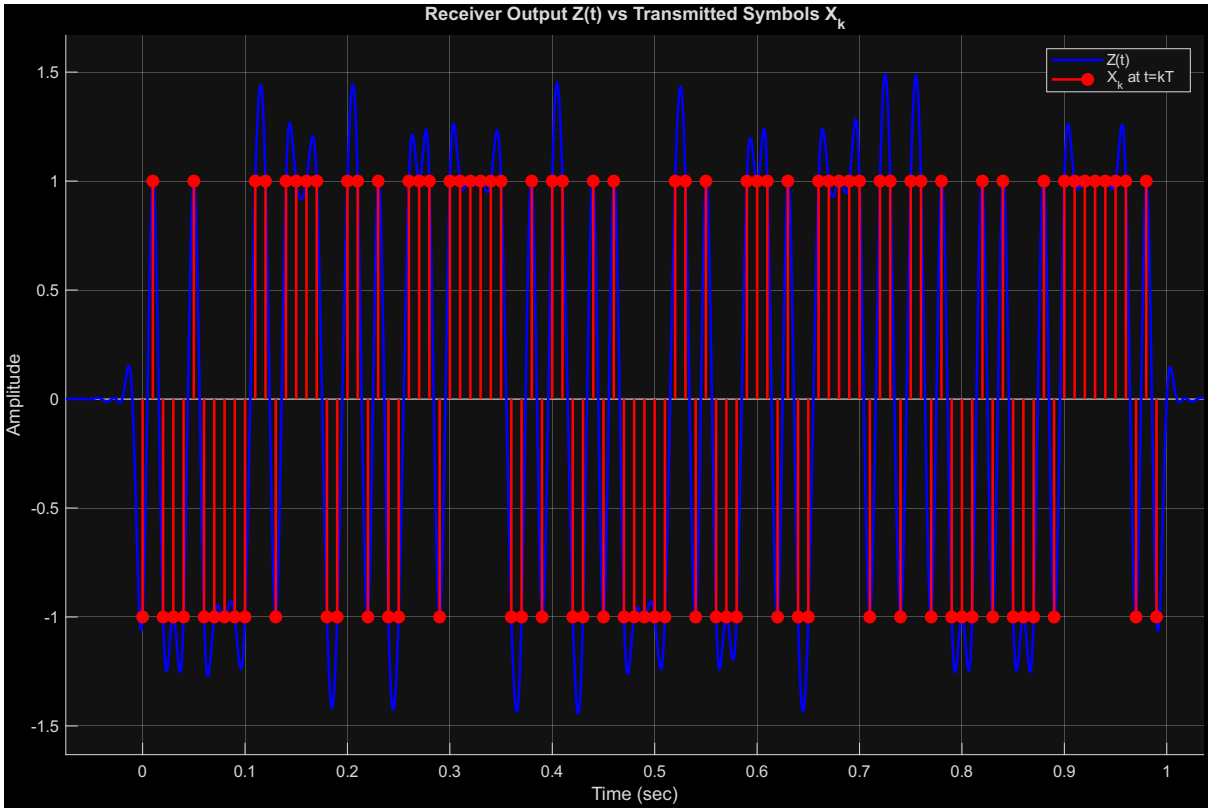

Figure 9: Constructed signal $Z(t)$ from convolution.)

MATLAB Code:

```
% C.1
N = 100;
b = (sign(randn(N, 1)) + 1) / 2;

% C.2.1
X = bits_to_2PAM(b);

% C.2.2
X_up = zeros(1, length(X) * over); %Upsample sequence
X_up(1:over:end) = X; % Place symbols every T seconds

X_delta = (1/Ts) * X_up; % Scale by 1/Ts to approximate Dirca Impulses
t_delta = (0 : length(X_delta) - 1) * Ts;
figure();
stem(t_delta, X_delta);
title('Simulated Impulse Train X_\delta(t)');
xlabel('Time (sec)');
ylabel('Amplitude');
grid on;

% C.2.3
[pulse, t] = srrc_pulse(T, over, A, a(2));
X_t = conv(X_delta, pulse) * Ts;
t_X = t_delta(1) + t(1) + (0:length(X_t)-1) * Ts;
figure();
plot(t_X, X_t);
title('Transmitted Signal X(t)');
xlabel('Time (sec)');
ylabel('Amplitude');
grid on;
```

```
% C.2.4
pulse_matched = fliplr(pulse); %phi(-t)
t_matched = -fliplr(t); %flipped time vector

Z_t = conv(X_t, pulse_matched) * Ts; %Receiver output
t_Z = t_X(1) + t_matched(1) + (0:length(Z_t) -1 ) * Ts;

k_vals = 0:(N-1);
sampling_times = k_vals * T; %Sampling Instants

%Z_sampled = interp1(t_Z, Z_t, sampling_times, 'nearest');

figure();
hold on;
plot(t_Z, Z_t, 'b', 'LineWidth', 1.5, 'DisplayName', 'Z(t)');
stem(sampling_times, X, 'r', 'filled', 'LineWidth', 1.5, 'DisplayName', '
X_k at t=kT');
title('Receiver Output Z(t) vs Transmitted Symbols X_k');
xlabel('Time (sec)');
ylabel('Amplitude');
legend('show');
grid on;
xlim([min(sampling_times)-T, max(sampling_times)+T]);
hold off;

```

Explanation: Comparing the sampled values $Z(kT)$ with the original transmitted symbols X_k, we observe a perfect match. This occurs because convolving the transmit SRRC filter $\phi(t)$ with the receiver matched SRRC filter $\phi(-t)$ yields an overall system response equivalent to a Raised Cosine (RC) pulse. The RC pulse satisfies the Nyquist criterion for zero Inter-Symbol Interference, meaning the contribution of all adjacent symbols at the exact sampling instant $t = kT$ is precisely zero.